ROUTEEV

RouteEV — Documento técnico

Referencia técnica completa del sistema: arquitectura, flujo de datos, API REST, algoritmos de ruteo y seguridad. Para la presentación general del proyecto ver `README.md`, para la guía de despliegue `DEPLOY.md`.

Arquitectura, algoritmos e implementación

Índice

1. Arquitectura general
2. Stack tecnológico
3. Modelo de datos
4. Flujo end-to-end
5. Referencia de la API
6. Algoritmos de ruteo
7. Motor de datos geográficos
8. Seguridad
9. Testing
10. Decisiones de diseño
11. Limitaciones y trabajo futuro

Proyecto de tesis · Uruguay

Generado el 31/07/2026 a partir de `TECHNICAL.md`

github.com/mqtissj/routesd

Arquitectura general

RouteEV es una aplicación monolítica de tres capas: un frontend estático sin build, una API REST en FastAPI, y un grafo vial completo cargado en memoria una única vez al arrancar el proceso.

```
flowchart LR
    subgraph Cliente["Navegador"]
        UI["HTML/CSS/JS plano<br/>Leaflet.js"]
    end

    subgraph Servidor["FastAPI (Uvicorn, un proceso)"]
        MW["Middleware de seguridad<br/>(CSP, headers, limite de body)"]
        API["Routers: auth / vehicles / graph / route"]
        GRAPH["GRAPH en memoria<br/>(Graph: ~63.800 nodos, ~97.300 aristas)"]
        ALGO["dijkstra.py / astar.py<br/>+ ev_logic.py"]
        DB[("SQLite<br/>routeev.db")]
    end

    subgraph Externos["Servicios externos"]
        OSM["OpenStreetMap<br/>Overpass API (offline, solo al regenerar el grafo)"]
        NOM["Nominatim<br/>(geocodificacion de direcciones)"]
        GOOGLE["Google Identity<br/>(login opcional)"]
        ANTHROPIC["API de Anthropic<br/>(busqueda de EV por IA, opcional)"]
        CARTO["CartoDB<br/>(tiles del mapa)"]
    end

    UI -- "fetch() + JWT Bearer" --> MW --> API
    API --> GRAPH
    API --> ALGO
    API --> DB
    API -. runtime .-> NOM
    API -. runtime .-> GOOGLE
    API -. runtime .-> ANTHROPIC
    UI -. tiles .-> CARTO
    OSM -. "generacion offline<br/>(fetch_osm_graph.py)" .-> GRAPH
```

Puntos clave:

- **Un solo proceso, un solo worker.** El grafo se carga entero en memoria (`backend/app/state.py`) al importar el módulo; con varios workers de Uvicorn cada uno tendría su propia copia (~104 MB cada una — ver `DEPLOY.md`).
- **Sin build de frontend.** HTML/CSS/JS servidos directamente como archivos estáticos (`StaticFiles` de FastAPI, con `Cache-Control: no-cache` para que el navegador siempre revalide contra el servidor).
- **Los datos geográficos son estáticos en tiempo de ejecución.** El grafo y las estaciones de carga se generan **offline** con scripts (`backend/scripts/`) contra Overpass/Nominatim y quedan guardados como JSON en `backend/data/` ; el servidor no vuelve a golpear esas APIs para servir el mapa, solo para geocodificar direcciones escritas a mano en tiempo real.

Stack tecnológico

ÁREA	TECNOLOGÍA	ROL
Framework web	FastAPI 0.140 + Uvicorn 0.51	API REST, servidor ASGI, documentación OpenAPI automática (solo en desarrollo)
Validación	Pydantic 2.13	Esquemas de entrada/salida (`schemas.py`), validación automática de tipos/rangos
Autenticación	PyJWT 2.13, `hashlib` / `hmac` (stdlib)	Tokens de sesión (HS256), hash de contraseñas (PBKDF2-HMAC-SHA256)
Persistencia	SQLite vía `sqlite3` (stdlib)	Usuarios, vehículos, historial de rutas — sin ORM, SQL explícito y parametrizado
Config	`python-dotenv` 1.2	Carga de `.env` en desarrollo
Cripto	`cryptography` 49.0	Dependencia de PyJWT para verificación de firmas
Testing	pytest 9.1, `httpx` (vía `TestClient`)	18 tests: algoritmos + regresión de seguridad
Frontend	HTML/CSS/JS vanilla (sin framework, sin bundler)	4 páginas (`landing` , `login` , `app` , `profile`), JS modular por página
Mapa	Leaflet.js (CDN `unpkg.com`)	Renderizado del grafo, marcadores, polilíneas de ruta
Tiles	CartoDB `dark_all` (CDN `basemaps.cartocdn.com`)	Cartografía base en tema oscuro
Datos viales	OpenStreetMap vía Overpass API	Fuente del grafo (offline, scripts de generación)
Geocodificación	Nominatim (OpenStreetMap)	Direcciones escritas a mano → coordenadas (runtime)
Datos de carga	UTE Movilidad (listado público)	209 estaciones reales, parseadas y geocodificadas offline
Login social	Google Identity Services + JWKS	Verificación de ID token contra las claves públicas de Google
IA opcional	API de Anthropic (`claude-haiku-4-5`)	Identificación de modelos de EV no catalogados, vía HTTP directo (sin SDK)

ÁREA	TECNOLOGÍA	ROL
CI / seguridad	GitHub Actions, Semgrep (`p/security-audit` , `p/owasp-top-ten` , `p/secrets` , `p/python` , `p/javascript`)	Escaneo estático + tests en cada push y semanalmente

Por qué no hay build de frontend ni framework de JS: con 4 páginas y sin necesidad de componentes reactivos complejos, un bundler agrega una capa de configuración y de `node_modules` sin beneficio claro para un MVP académico. El costo es más verbosidad en `map.js` (el archivo más grande del frontend); el beneficio es que el proyecto entero corre con un único `pip install` , sin Node.

Por qué SQLite y no un motor cliente-servidor: ver "Decisiones de diseño" más abajo.

Modelo de datos

Tres tablas, todas en `backend/app/database.py` , con SQL explícito (sin ORM) y migraciones manuales vía `ALTER TABLE` idempotente (`_migrate()`):

```
erDiagram
    USERS ||--o{ VEHICLES : "posee"
    USERS ||--o{ ROUTE_HISTORY : "calculo"
    VEHICLES ||--o{ ROUTE_HISTORY : "usado en"

    USERS {
        int id PK
        text username UK
        text password_hash
        text email
        text google_sub UK "nullable"
        text created_at
    }
    VEHICLES {
        int id PK
        int user_id FK
        text name
        real battery_capacity_kwh
        real consumption_kwh_per_km
        real charging_power_kw
        text created_at
    }
    ROUTE_HISTORY {
        int id PK
        int user_id FK
        int vehicle_id FK
        text origin_id
        text destination_id
        text origin_name
        text destination_name
        real distance_km
        real energy_kwh
        real trip_time_min
        int recharge_count
        real toll_cost
        real costo_uyu_total
        text result_json "ruta completa + breakdown"
        text created_at
    }
```

Todas las consultas que tocan `vehicles` o `route_history` filtran explícitamente por `user_id` (ver `database.get_vehicle`, `delete_vehicle`, `list_route_history`, `monthly_trip_stats`): no existe un endpoint que resuelva un `vehicle_id` o un registro de historial sin cruzarlo contra el usuario autenticado, lo que

evita accesos de un usuario a datos de otro (IDOR) por diseño.

El grafo vial (`seed_graph.json`) y las estaciones (`charging_stations.json`) **no** viven en la base de datos: son archivos JSON generados offline y cargados enteros en memoria al arrancar (ver `graph.py: Graph.load`), porque son de solo lectura durante la vida del proceso y compartidos por todos los usuarios.

Flujo end-to-end

Secuencia completa desde que un usuario nuevo entra hasta que calcula una ruta y la sigue en vivo:

```
sequenceDiagram
    actor U as Usuario
    participant FE as Frontend (login.js / map.js)
    participant API as FastAPI
    participant DB as SQLite
    participant G as GRAPH (memoria)
    participant NOM as Nominatim

    U->>FE: Completa usuario/contraseña, "Crear cuenta"
    FE->>API: POST /api/auth/register
    API->>DB: INSERT users (password_hash = PBKDF2)
    API-->>FE: JWT (access_token)
    FE->>FE: localStorage.setItem(token)
    FE->>U: Redirige a /app

    FE->>API: GET /api/graph  (Authorization: Bearer JWT)
    API-->>FE: {nodes, edges} (~1.6 MB gzip)
    FE->>FE: Dibuja el grafo con Leaflet

    U->>FE: "+ Nuevo vehiculo" -> elige modelo -> "Crear vehiculo"
    FE->>API: POST /api/vehicles
    API->>DB: INSERT vehicles
    API-->>FE: VehicleResponse

    alt Origen/destino por click en el mapa
        U->>FE: Click en el mapa
        FE->>API: POST /api/graph/nearest {lat, lon}
        API-->>FE: nodo real mas cercano
    else Direccion escrita a mano
        U->>FE: Escribe direccion, "Buscar"
        FE->>API: GET /api/graph/geocode?q=...
        API->>G: Intenta resolver contra el grafo propio (interseccion)
        API->>NOM: (solo si hace falta) geocode_address()
        API-->>FE: nodo real mas cercano + distancia
    end

    U->>FE: "Calcular ruta"
    FE->>API: POST /api/route/compare {origin_id, destination_id, vehicle_id, start_battery_kwh}
    API->>G: astar.search(...)  [+ dijkstra.search(...) si comparar=true]
    G-->>API: RouteResult (ruta, recargas, breakdown de costo)
    API-->>FE: {origin, destination, astar, dijkstra}  (no escribe en la base)
    FE->>U: Panel "Datos del viaje" + ruta resaltada en el mapa
```

```
    U->>FE: "Iniciar viaje"
    FE->>FE: navigator.geolocation.watchPosition(...)
    loop cada actualizacion de posicion GPS real
        FE->>FE: Proyecta la posicion sobre route_coords
        FE->>U: Distancia recorrida, bateria estimada, alerta si se desvia
    end

    alt Llega a destino, o cancela habiendo avanzado
        FE->>API: POST /api/route/log-trip {origen, destino, vehiculo, traveled_km, recharge_node_ids}
        API->>G: astar.search(...) de nuevo + truncate_route(traveled_km)
        API->>DB: INSERT route_history (tramo REAL recorrido)
    else Cancela sin haber avanzado
        FE->>FE: No se llama a ningun endpoint -- nada queda guardado
    end

    U->>FE: "Mi perfil"
    FE->>API: GET /api/route/history + GET /api/route/monthly-stats
    API->>DB: SELECT ... WHERE user_id = ?
    API-->>FE: Historial completo + promedios mensuales
```

Nota sobre "Iniciar viaje": la implementación actual (`frontend/js/map.js` , función `startLiveTrip`) sigue la **ubicación real del dispositivo** vía `navigator.geolocation.watchPosition` , proyectando cada posición GPS sobre la polilínea de la ruta ya calculada para derivar distancia recorrida, batería consumida y llegada a paradas de recarga — no es una animación simulada con un cronómetro. Un ícono de auto (`live-location-car`) queda apilado sobre el punto/pulso de posición, así que se mueve pegado a cada actualización real de GPS. Si el navegador deniega el permiso de ubicación, la interfaz lo informa explícitamente en vez de fallar en silencio (`handleLiveError`).

El historial refleja viajes reales, no cálculos. `POST /api/route/compare` solo calcula y muestra — no escribe en la base. El historial (`route_history`) se llena exclusivamente vía `POST /api/route/log-trip` , que el frontend llama en dos momentos: al llegar físicamente a destino, o al cancelar el seguimiento en vivo habiendo recorrido una distancia real mínima (`MIN_PROGRESS_TO_LOG_KM = 0.03 km` , para filtrar jitter de GPS quieto). Cancelar sin haber avanzado no deja ningún rastro. El backend recalcula la ruta óptima origen→destino y la recorta al prefijo de nodos que corresponde a la distancia realmente recorrida (`route_result.truncate_route`), así que un viaje cortado a mitad de camino queda guardado con su distancia, energía, tiempo y peajes propios — no los del viaje completo que se había planeado — y con el **cruce real donde se detuvo** como destino guardado, no el destino original tipeado. Ver el docstring de `route_routes.py: log_trip` para el detalle completo.

Referencia de la API

Todos los endpoints devuelven JSON y (salvo los marcados como públicos) exigen `Authorization: Bearer <JWT>`. Los de escritura o intensivos en cómputo/red están detrás de un limitador de frecuencia por IP o por usuario (ver Seguridad).

Autenticación — `/api/auth`

MÉTODO Y RUTA	AUTH	LÍMITE	DESCRIPCIÓN
`GET /api/auth/config`	Pública	—	Config pública: si Google login y búsqueda por IA están habilitados en este servidor
`POST /api/auth/register`	Pública	10/5 min por IP	`{username, password}` → crea cuenta (password ≥ 10 caracteres) y devuelve JWT
`POST /api/auth/login`	Pública	10/5 min por IP	`{username, password}` → JWT. Tiempo de respuesta constante exista o no el usuario
`POST /api/auth/google`	Pública	10/5 min por IP	`{id_token}` (Google Identity Services) → verifica contra JWKS de Google y devuelve JWT propio

Vehículos — `/api/vehicles`

MÉTODO Y RUTA	AUTH	LÍMITE	DESCRIPCIÓN
`GET /api/vehicles`	JWT	—	Lista los vehículos del usuario autenticado
`GET /api/vehicles/models`	JWT	—	Catálogo estático de 880 modelos de EV reales (14 curados para Uruguay + 866 de un dataset global)
`POST /api/vehicles/lookup-ai`	JWT	20/hora por usuario	`{query}` → identifica un EV por nombre vía Claude; 503 si no está configurado, 404 si no es un EV real
`POST /api/vehicles`	JWT	—	Crea un vehículo (`name` , `battery_capacity_kwh` , `consumption_kwh_per_km` , `charging_power_kw`)
`DELETE /api/vehicles/{vehicle_id}`	JWT	—	Borra un vehículo propio; 404 si no existe o no es del usuario

Grafo — `/api/graph`

MÉTODO Y RUTA	AUTH	LÍMITE	DESCRIPCIÓN
`GET /api/graph`	JWT	30/hora por usuario	Grafo completo (nodos + aristas) para dibujar el mapa (~1.6 MB comprimido)
`GET /api/graph/stations`	JWT	—	Las 209 estaciones de carga con metadata completa
`GET /api/graph/searchable-nodes?limit=`	JWT	400/hora por usuario	Subconjunto representativo de intersecciones para un selector tipo lista
`POST /api/graph/nearest`	JWT	400/hora por usuario	`{lat, lon}` → nodo real más cercano (click en el mapa / "usar mi ubicación"). POST a propósito: una coordenada real no debe quedar en la URL ni en logs
`GET /api/graph/geocode?q=`	JWT	30/hora por usuario*	Dirección escrita a mano → nodo del grafo. Resuelve primero contra intersecciones propias del grafo; solo golpea Nominatim si hace falta

* El límite de geocodificación solo se consume cuando la consulta efectivamente sale a Nominatim — si se resuelve contra el grafo propio, no se gasta cupo.

Rutas — `/api/route`

MÉTODO Y RUTA	AUTH	LÍMITE	DESCRIPCIÓN
`POST /api/route/compare`	JWT	60/hora por usuario	`{origin_id, destination_id, vehicle_id, start_battery_kwh?, comparar?}` → calcula la ruta con A *(y con Dijkstra si* `comparar=true` *) y devuelve ambos resultados. Solo calcula: no guarda nada en el historial**
`POST /api/route/log-trip`	JWT	60/hora por usuario	`{origin_id, destination_id, vehicle_id, start_battery_kwh?, traveled_km, recharge_node_ids?}` → guarda en el historial el tramo **realmente recorrido** de un viaje en vivo (llegada a destino, o cancelación habiendo avanzado). Recorta la ruta óptima al prefijo que corresponde a `traveled_km`; devuelve el nodo real donde se detuvo el viaje. 400 si `traveled_km` es tan chico que no cubre ni una arista
`GET /api/route/history`	JWT	—	Últimos 20 viajes **realmente recorridos** por el usuario (no cálculos de ruta sin más)
`GET /api/route/monthly-stats`	JWT	—	Promedios mensuales (viajes, distancia, energía, tiempo, costo) agrupados por mes, sobre los viajes del historial

Páginas y utilidad

RUTA	DESCRIPCIÓN
`GET /`	Landing page
`GET /login`	Login / registro
`GET /app`	Mapa (requiere sesión en el frontend)
`GET /profile`	Perfil e historial (requiere sesión en el frontend)
`GET /api/health`	Health check simple `{"status": "ok"}`
`GET /docs` , `/openapi.json`	Documentación interactiva de la API — **solo en desarrollo**, apagada en producción

Algoritmos de ruteo

Función de costo compuesto

Cada arista del grafo (`graph.py: Edge.cost`) tiene un único costo, en **minutos equivalentes**, que combina cuatro componentes:

```
costo_arista = tiempo_min
             + ENERGY_WEIGHT_MIN_PER_KWH * energia_kwh(vehiculo)
             + penalizacion_trafico
             + penalizacion_peaje
```

`ENERGY_WEIGHT_MIN_PER_KWH` (default `5.0` , en `config.py`) es el "tipo de cambio" entre energía y tiempo: cuántos minutos-equivalente le cuesta al algoritmo gastar 1 kWh de batería. En `0` , el sistema se comporta como un ruteo de "más rápido a secas"; subirlo hace que las rutas prioricen más el ahorro de batería aunque tarden más. La energía depende del vehículo consultado (autos distintos consumen distinto por km), así que **no** viene precalculada en el JSON del grafo: se calcula al vuelo contra el vehículo de cada consulta.

Espacio de estados: por qué "el nodo" no alcanza

Un Dijkstra/A *de libro guarda un único costo óptimo por nodo*. Con restricción de batería eso falla: dos caminos distintos pueden llegar al mismo nodo con costo y batería distintos — uno más barato pero con poca batería restante (que después puede quedar sin salida), y otro más caro pero con más batería (que sí puede seguir viaje o llegar a destino). Si solo se guardara "el mejor costo por nodo", el camino barato-pero-sin-batería cerraría ese nodo para siempre y descartaría la alternativa que sí podía continuar — el algoritmo podía reportar "sin ruta" existiendo una.

La solución (`dijkstra.py` , `astar.py`) es que el **estado de búsqueda sea el par** `(nodo, nivel_de_batería)` , no solo el nodo. El nivel de batería se discretiza en `config.BATTERY_BUCKETS` niveles (por defecto 20) para acotar el espacio de estados; el nivel exacto se sigue guardando con precisión

completa en `battery_exact` — el bucket solo decide si dos llegadas al mismo nodo son "lo bastante distintas" en batería como para tratarse como estados separados.

La recarga en una estación es una **transición más del grafo aumentado**: del estado `(nodo_estación, bucket_bajo)` al estado `(nodo_estación, bucket_lleno)`, con costo igual al tiempo de recarga estimado. No es un paso forzado — el algoritmo la explora como una opción más y solo "gana" si el costo total resulta menor, igual que cualquier otra arista.

Dijkstra (baseline)

Implementación clásica de cola de prioridad (`heapq`) sobre el espacio de estados ampliado: se expande siempre el estado de menor `g` (costo acumulado real) hasta llegar al nodo destino. Sirve como referencia de corrección: si A* devuelve un costo distinto a Dijkstra para el mismo origen/destino/vehículo, hay un bug en la heurística.

A* (algoritmo principal)

Estructuralmente idéntico a Dijkstra, con una única diferencia: la cola de prioridad se ordena por `f(n) = g(n) + h(n)` en vez de solo por `g(n)`, donde:

```
h(n) = haversine_km(n, destino) / velocidad_maxima_del_grafo * 60
```

Por qué `h(n)` es admisible (nunca sobreestima el costo real restante — condición necesaria para que A* siga garantizando la ruta óptima):

Para cualquier arista real del grafo:

```
costo_arista = tiempo_min + ENERGY_WEIGHT * energia_kwh + trafico + peaje
             ≥ tiempo_min                                  (los otros 3 terminos son ≥ 0)
             ≥ distancia_km / velocidad_arista * 60          (definicion de tiempo_min)
             ≥ distancia_km / velocidad_maxima_grafo * 60     (velocidad_arista ≤ velocidad_maxima_grafo)
```

Sumando esta desigualdad arista por arista a lo largo de **cualquier** camino real entre `n` y el destino (incluidas eventuales paradas de recarga, que solo suman costo), el costo total del camino es siempre `≥ suma_de_distancias / velocidad_maxima_grafo * 60`. Por desigualdad triangular, la suma de distancias de cualquier camino es siempre `≥` la distancia en línea recta entre sus extremos, así que el costo de cualquier camino entre `n` y el destino es siempre `≥ h(n)`. Como esto vale en particular para el camino **óptimo**, `h(n)` nunca sobreestima el costo real restante: es **admisible**. Al cumplirse arista por arista (no solo en el agregado), también es **consistente/monótona**, la propiedad que permite usar un closed-set (como en Dijkstra) sin perder optimalidad.

La heurística depende **únicamente** de la posición física del nodo, nunca de la batería: ignora por completo la restricción energética. Esto no rompe la admisibilidad — al contrario, la refuerza, porque una restricción de batería solo puede obligar a rodeos o paradas de recarga que **aumentan** el costo real, nunca a bajarlo.

Resultado verificado empíricamente (`test_algorithms.py`): Dijkstra y A* devuelven siempre el mismo costo óptimo sobre el grafo real, con y sin restricción de batería, en decenas de pares origen/destino aleatorios; A* nunca expande más nodos que Dijkstra.

Modelo de vehículo eléctrico

`ev_logic.py` — funciones puras, sin estado:

- **Consumo**: `energia_kwh = distancia_km * consumption_kwh_per_km` (específico de cada vehículo).
- **Tiempo de recarga**: `(batería_faltante_kwh / potencia_efectiva_kw) * 60` , donde la potencia efectiva es el **mínimo** entre lo que el vehículo puede aceptar y lo que la estación puede entregar — un auto de carga rápida a 250 kW igual tarda lo que tarda si la estación solo entrega 22 kW, y viceversa.
- **Verificación de batería**: antes de relajar cualquier arista, se comprueba que la batería actual alcance para el consumo de esa arista con ese vehículo; si no alcanza, la arista se descarta para ese estado de búsqueda (no para el nodo en general — otro estado con más batería sí puede tomarla).

Motor de datos geográficos

- **Generación del grafo** (`backend/scripts/fetch_osm_graph.py`): pega directo a la API Overpass (HTTP + JSON) y simplifica intersecciones a mano — sin OSMnx, por una restricción de Application Control en el entorno de desarrollo original (ver el script para el detalle). Produce `seed_graph.json` con nodos, aristas, forma real de cada calle (`shape`) y metadata de peajes (`toll=yes` de OSM).
- **Cobertura en dos niveles**: Montevideo urbano con calles residenciales incluidas; los otros 18 departamentos solo con vías principales (rutas nacionales, avenidas primarias), más un radio fino (1,5 km) alrededor de cada estación de carga fuera de la capital para que quede conectada a la red real y no por una arista sintética en línea recta.
- **Resolución de intersecciones por texto** (`graph.py: find_intersection_candidates`): las direcciones uruguayas se describen por cruce ("18 de Julio y Rivera"), no por numeración; el matching normaliza acentos, expande abreviaturas comunes (`Av.` → `avenida`), descarta *stopwords* (`de` , `la` , ...) y exige que **todas** las palabras significativas de cada calle estén presentes — evita falsos positivos como resolver "18 de Julio y Uruguay" a un cruce de otro departamento que solo compartía la palabra "de". Cuando el mismo nombre de cruce existe en varias ciudades (común en Uruguay), usa la ciudad mencionada después de una coma como ancla de desempate contra Nominatim.
- **Selector de intersecciones** (`GET /api/graph/searchable-nodes`): con ~63.800 nodos, ofrecer "todos" en una lista no es usable. Se garantiza un mínimo por celda de una grilla geográfica gruesa (cobertura regional) y el resto del cupo se llena con las de mayor grado globalmente (que naturalmente prioriza cruces importantes de Montevideo sin excluir el resto del país).
- **Estaciones de carga como nodos del grafo** (`graph.py: _merge_charging_stations`): cada estación se conecta con una arista corta bidireccional al cruce real más cercano (15 km/h fijo, sin penalización de tráfico/peaje — es un tramo de acceso, no una vía).

Seguridad

Esta sección documenta tanto las defensas **implementadas** como la **auditoría de seguridad realizada sobre este código** (no solo una descripción de intenciones).

Auditoría realizada

- **Suite de regresión de seguridad**: 12 tests dedicados (`backend/tests/test_security.py`) que verifican, contra la app real vía `TestClient` , que las defensas de abajo sigan en pie — 18/18 tests (algoritmos + seguridad) pasan.
- **Análisis estático con Semgrep**: 356 reglas (`p/security-audit` , `p/owasp-top-ten` , `p/secrets` , `p/python` , `p/javascript`) sobre los 54 archivos versionados del repositorio (backend Python + frontend JS/HTML) — **0 hallazgos**.
- **Revisión manual dirigida** de autenticación (`auth.py`), autorización a nivel de fila (`database.py`), rate limiting (`rate_limit.py`) y las rutas de escritura (`vehicle_routes.py` , `route_routes.py` , `auth_routes.py`): sin patrones de IDOR (toda consulta a `vehicles` / `route_history` filtra por `user_id`), sin SQL sin parametrizar, sin `eval` /deserialización insegura, y las URLs de servicios externos (Nominatim, Anthropic) son constantes fijas — el texto que escribe el usuario viaja solo en la query string o el cuerpo JSON, nunca puede cambiar esquema/host de la petición saliente.
- **Prueba funcional end-to-end** (login, registro, creación de vehículo, cálculo de ruta interdepartamental con peaje, seguimiento en vivo, perfil, casos límite de validación) contra el servidor real corriendo localmente: encontró y corrigió una inconsistencia de redirección (`frontend/js/profile.js` mandaba a `/` en vez de `/login` cuando no había sesión activa, a diferencia de `map.js`).

Defensas implementadas

RIESGO	DEFENSA	DÓNDE
Fuerza bruta de contraseñas	PBKDF2-HMAC-SHA256, 260.000 iteraciones; mínimo 10 caracteres al registrarse	`auth.py`, `schemas.py`
Enumeración de usuarios por temporización	Hash descartable comparado siempre que el usuario no existe, mismo costo que una comparación real	`auth.py: verify_password_dummy`
Fuerza bruta de login/registro	10 intentos / 5 min por IP	`rate_limit.py`, `auth_routes.py`
Gasto ilimitado en la API de IA	20 consultas/hora por usuario	`config.py`, `vehicle_routes.py`
Saturación de CPU (ruteo)	60 cálculos/hora por usuario	`route_routes.py`
Bloqueo de la IP del servidor en OSM	30 geocodificaciones/hora por usuario, solo se consume si hace falta salir a Nominatim	`graph_routes.py`
Token falsificado / alterado	JWT firmado HS256, secreto de 64 bytes fuera del repo; en producción el arranque falla si falta `JWT_SECRET`	`config.py`, `auth.py`
Token sin expiración aceptado para siempre	`jwt.decode(..., options={"require": ["exp", "sub"]})`	`auth.py: get_current_user`
Acceso a datos de otro usuario (IDOR)	Toda consulta a `vehicles` / `route_history` incluye `WHERE user_id = ?`	`database.py`
Inyección SQL	100% de las consultas parametrizadas (`sqlite3` con placeholders `?`), sin concatenación de strings de usuario	`database.py`
XSS almacenado/reflejado	`textContent` (no `innerHTML`) para todo dato ingresado por el usuario en el frontend; nombres de usuario limitados a `[A-Za-z0-9._-]`	`*.js`, `schemas.py`
Clickjacking	CSP `frame-ancestors 'none'` + `X-Frame-Options: DENY`	`main.py`
MIME sniffing	`X-Content-Type-Options: nosniff`	`main.py`
Downgrade a HTTP	`Strict-Transport-Security` (solo en producción)	`main.py`
Fuga del punto exacto del usuario en logs	`POST /api/graph/nearest` con coordenada en el cuerpo, nunca en query string	`graph_routes.py`, `schemas.py`
Body de request desproporcionado	Corte por `Content-Length` antes de procesar (64 KB)	`main.py`
CORS mal configurado	Deshabilitado por defecto (mismo origen sirve API y frontend); nunca `*` si se activa	`config.py`, `main.py`

RIESGO	DEFENSA	DÓNDE
Spoofing de IP para saltear rate limit	Headers `X-Forwarded-For` / `X-Real-IP` solo se confían si `TRUST_PROXY_HEADERS=1` (deploy detrás de proxy real)	`rate_limit.py`
Reconocimiento gratuito de la API	`/docs` , `/redoc` , `/openapi.json` apagados en producción	`main.py`
Secreto de sesión filtrado en el pasado	Uno anterior quedó commiteado; se considera comprometido y hay que rotarlo antes de desplegar (ver `DEPLOY.md`)	—

Login con Google

`auth.py: verify_google_id_token` valida el ID token contra las claves públicas de Google (JWKS, cacheadas vía `PyJWT.PyJWKClient`), exige `audience = GOOGLE_CLIENT_ID` e `issuer = accounts.google.com` . El email solo se usa para derivar un nombre de usuario si Google lo reporta como verificado (`email_verified`); si no, la cuenta se crea con un nombre derivado del `sub` (identificador estable de la cuenta de Google, siempre confiable porque viene de un token ya verificado).

Limitación conocida del rate limiting

El estado de los límites de frecuencia vive en memoria del proceso (`rate_limit.py`). Con un solo worker Uvicorn (como corre hoy) es correcto; al escalar a varios workers o instancias, cada uno llevaría su propia cuenta y el límite real terminaría siendo `N × configurado` . Para eso, `rate_limit.py` documenta la migración a un almacén compartido (Redis) o al limitador del proxy/CDN de adelante.

Testing

```
pytest -v
```

ARCHIVO	TESTS	QUÉ VERIFICA
`backend/tests/test_algorithms.py`	6	Dijkstra y A *devuelven el mismo costo óptimo (con y sin restricción de batería), la heurística nunca sobreestima el costo real, A* nunca expande más nodos que Dijkstra, las rutas son caminos válidos del grafo, y una batería imposible reporta "sin ruta" sin crashear
`backend/tests/test_security.py`	12	Rechazo de tokens con secreto viejo/sin expiración/ `sub` no numérico, endpoints privados exigen auth, contraseñas cortas rechazadas, rate limit de login, headers de seguridad presentes, `Permissions-Policy` no bloquea la geolocalización propia, todos los endpoints caros tienen tope, geocodificación corta antes de golpear OSM, la ubicación no viaja en la URL, bodies desproporcionados rechazados

CI (`.github/workflows/security.yml`) corre Semgrep (contenedor Linux) y pytest en cada push, cada pull request, y cada lunes 6:00 UTC.

Decisiones de diseño

- **Landing + login separados del mapa**: `/` es una página de presentación (el problema, la solución, datos reales del grafo), no un formulario. El login vive en `/login` , y el mapa/app en `/app` , detrás de sesión — separa el "por qué" (para quien no conoce el proyecto) del "cómo" (para quien ya lo va a usar).
- **"Datos del viaje" como resultado principal, no la comparación de algoritmos**: la comparación Dijkstra/A* sigue implementada y probada (`test_algorithms.py` , `POST /api/route/compare` con `comparar: true`), pero el resultado que le importa a quien usa la app es el viaje en sí, no qué algoritmo lo calculó.
- **La comparación es opcional, no automática**: correr los dos algoritmos en cada consulta duplicaba la espera para mostrar el mismo resultado óptimo (medido: ~5,9 s con los dos, ~2,9 s con solo A *sobre el grafo de esa medición). Por defecto solo corre A*.
- **SQLite en vez de un motor cliente-servidor**: para una BETA académica que corre localmente, SQLite no depende de una cuenta externa ni de conexión a internet para login/vehículos/historial. Si el proyecto necesita acceso multi-dispositivo o en la nube, `database.py` es el único módulo a reemplazar (las consultas ya son explícitas y acotadas).
- **Nominatim en vez de un proveedor de mapas pago**: gratis, sin API key, mismo proveedor de datos que ya se usa para el grafo vial.
- **Catálogo de EV embebido + búsqueda por IA opcional**: evita depender de una API paga/con key para lo que en esencia es un archivo JSON estático (`ev_models.json` , hoy 880 modelos: 14 curados para Uruguay con notas de contexto, más 866 de un dataset EV global sin curar línea por línea), con una salida de emergencia (Claude) para modelos fuera del catálogo. El selector de modelo es un `<input list>` + `<datalist>` en vez de un `<select>` : con 880 opciones un select deja de ser navegable, pero el autocompletado nativo de `<datalist>` filtra en vivo lo que se tipea sin agregar ningún componente propio de JS (busca por texto exacto una vez que el usuario confirma un modelo real).
- **Grafo real vía Overpass en vez de un grafo pequeño armado a mano**: para poder cubrir el país completo con datos viales reales en vez de una simplificación didáctica.
- **Dos niveles de detalle geográfico** (Montevideo fino / resto del país solo vías principales): cubrir los 19 departamentos con el mismo detalle que una ciudad generaría un grafo de cientos de miles de nodos: el ruteo interdepartamental real casi siempre va por rutas nacionales, así que restringir la zona exterior a vías principales es una simplificación razonada.
- **El historial guarda viajes recorridos, no cálculos de ruta**: la primera versión guardaba una fila en `route_history` cada vez que se apretaba "Calcular ruta", aunque el usuario nunca llegara a manejarla — "calcular" es mirar el mapa, no hacer un viaje. La versión actual separa ambas cosas: `POST /api/route/compare` solo calcula, y `POST /api/route/log-trip` guarda el tramo real cuando el seguimiento en vivo por GPS termina (llegada a destino, o cancelación habiendo avanzado una distancia mínima). Cortar el viaje a mitad de camino sin haberse movido no deja rastro; cortarlo

habiendo avanzado guarda ese tramo real (distancia, energía, tiempo y peajes de lo efectivamente recorrido, no de la ruta completa que se había planeado) — la razón es que en el mundo real un viaje interrumpido a mitad de camino igual gastó batería real y vale la pena que quede reflejado en los promedios, mientras que un cálculo que nunca se manejó no debería contarse como un viaje.

Limitaciones y trabajo futuro

- **Tarifas de peaje**: se detecta el tramo con peaje real (`toll=yes` de OpenStreetMap, 39 tramos en el grafo actual), pero el monto en pesos (`Edge.peaje_uyu`) solo está cargado cuando se corrió `backend/scripts/apply_real_tolls.py` contra una fuente de tarifas verificadas — hoy ningún tramo tiene tarifa real cargada, así que el costo en pesos de los peajes se muestra como `$0` aunque la ruta sí los atraviese (esto es intencional y está señalizado en la interfaz, no un bug).
- **Precisión de coordenadas de estaciones**: los datos técnicos (potencia, conector, puertos) de las 209 estaciones son reales; la coordenada de las que no se pudieron geocodificar por dirección exacta queda a nivel de ciudad o departamento (campo `geocode_precision` en `charging_stations.json`).
- **Penalización de tráfico estática**: por tipo de vía, no en tiempo real.
- **Grafo exterior solo con vías principales**: no hay ruteo calle a calle dentro de ciudades del interior, solo hasta/desde sus rutas de acceso.
- **Escala limitada por el tamaño del grafo en memoria**: ~56 s de arranque y ~104 MB de RAM por proceso con el grafo actual (medido en `DEPLOY.md`); ambos números crecen si se regenera el grafo con más cobertura, y hay que remedirlos cada vez.
- `/api/graph` **es pesado** (~1,6 MB comprimido, todos los nodos/aristas del país): una mejora natural es cachear la respuesta con ETag o mandar solo lo que entra en el área visible del mapa.
- **Rate limiting en memoria de proceso**: no escala a varios workers/instancias sin migrar a un almacén compartido (ver Seguridad).
- **Sin recuperación de contraseña ni verificación de email**: es el login mínimo que necesita un MVP académico, no un sistema de autenticación de producción.